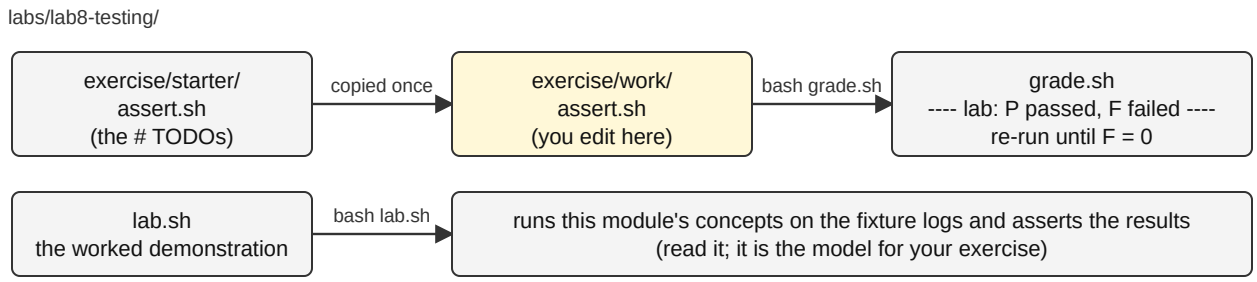
Lab 8 — Testing and Quality: The Smallest Useful Test Harness (Instructor Version)

Instructor Note: Gray boxes and answers marked in red appear in the instructor copy only.

Topology



Environment Table

All paths are relative to `labs/lab8-testing/exercise/`.

File	What it is	You edit it?
`TASK.md`	The short statement of the exercise	No
`starter/assert.sh`	`assert_eq` with its body as a `# TODO`	No — the grader copies it
`work/assert.sh`	**Your** copy (created by the first `grade.sh` run)	**Yes**
`grade.sh`	The grader: 4 checks. It sources your file, calls `assert_eq` three times and reads the counters	No
`work/REFLECTION.md`	Your two reflection answers	**Yes** (you create it)
`logsleuth/tests/assert.sh`	The course's own harness — the same idea, for comparison *after* you pass	No

Objectives

- **Part 1:** Prepare the exercise folder and read the grader's first report.
- **Part 2:** Build `assert_eq` — compare, count, report.
- **Part 3:** Verify with the grader; write the two reflection answers.

Background / Scenario

Every grader in this course, and `logsleuth`'s own test suite, is built on one function: compare what the code *produced* with what you *expected*, count passes and failures in two global variables, and print one line when they differ. That is the whole idea behind `bats`, xUnit and every test framework; the rest is convenience. In this lab you write that function:

`assert_eq NAME GOT WANT` — on a match, add 1 to the global `PASSED`; on a mismatch, add 1 to the global `FAILED` and print `FAIL NAME: got [GOT] want [WANT]`.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

The caller initialises `PASSED=0` and `FAILED=0`; your function must also behave when a caller forgot to.

Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready` (with `shellcheck`).
- Module M08, step S08.1.
- A text editor. Documentation and AI assistance for lookups are allowed (say so in your reflection). Do **not** open `solution/`.

> **Instructor Note:** Budget 45 minutes; grading ≈ 5 minutes per learner. Observed: the starter scores `---- lab8: 1 passed, 3 failed ----` (the empty function is already shellcheck-clean); the reference 4/4. The grader's third call is `assert_eq "mismatch" "1" "2" >/dev/null` — a redirect, not a `$( )` — precisely so the counters change in the caller's shell; Question 1 is about that.

Part 1: Prepare the exercise folder

Step 1: Run the grader once.

```
cd labs/lab8-testing/exercise
bash grade.sh
```

Expected: `---- lab8: 1 passed, 3 failed ----`.

Step 2: Open `work/assert.sh`.

One function, one `# TODO`, body `:` (a command that does nothing).

Part 2: Build `assert_eq`

Step 1: Compare, count, report.

Replace the `:` with:

```
assert_eq() {
  if [[ "$2" == "$3" ]]; then
    PASSED=$(( ${PASSED:-0} + 1 ))
  else
    FAILED=$(( ${FAILED:-0} + 1 ))
    printf 'FAIL %s: got [%s] want [%s]\n' "$1" "$2" "$3"
  fi
}
```

How it works: `$1` is the test's name, `$2` what the code produced (*got*), `$3` what you decided in advance (*want*). `[[ "$2" == "$3" ]]` compares the two strings exactly. `$(( ${PASSED:-0} + 1 ))` is **arithmetic** — not string concatenation — and the `:-0` makes it work even when a caller never set the counter. The counters are deliberately **global**: the whole point of the harness is that the caller reads them afterwards. A `local PASSED` would make every test disappear.

The brackets in `got [..] want [..]` are there to make leading or trailing spaces and empty strings visible.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 2: Try it by hand.

```
source work/assert.sh
PASSED=0; FAILED=0
double() { echo $(( $1 * 2 )); }
assert_eq "double 21" "$(double 21)" "42"
assert_eq "double 0"  "$(double 0)"  "99"      # a deliberate miss
printf 'passed=%d failed=%d\n' "$PASSED" "$FAILED"
```

Expected:

```
FAIL double 0: got [0] want [99]
passed=1 failed=1
```

Part 3: Verify and reflect

Step 1: Run the grader until it is green.

```
bash grade.sh
```

Expected: `---- lab8: 4 passed, 0 failed ----`. The checks: two matches count `PASSED=2`; one deliberate miss counts `FAILED=1`; the miss prints `got [0] want [99]`; `assert.sh` is shellcheck-clean.

Symptom	Cause	Fix
`counted two passes` got `unset`	The function assigns `passed` (lower case) or a `local`	Assign the globals `PASSED` / `FAILED`
`counted two passes` got `11`	String concatenation instead of arithmetic	`$(( ${PASSED:-0} + 1 ))`
The message check fails	Wording or spacing differs	Exactly `FAIL %s: got [%s] want [%s]`

Step 2: Reflection.

Create `work/REFLECTION.md` and answer in your own words (3–5 sentences each):

Question 1: The grader calls `assert_eq … >/dev/null` for the deliberate miss and still sees `FAILED=1`, but a call inside `$( )` would not change the counters. Explain where a test's *result* lives in this harness and why that place matters.

> **Answer:** **Met:** results live in the global counters `PASSED`/`FAILED`; a function called inside `$( )` runs in a subshell whose variable changes are discarded, while `>/dev/null` only redirects output and leaves the shell — and thus the counters — intact. **Strong:** connects it to Lab 3's piped-`while` subshell or to the grader's own `out="$( … )"` trick, or notes that printing the counts to stdout instead would force callers to parse text. **Not met:** "the message goes to /dev/null but the variable stays" without naming the subshell.
>
> *Calibration.* Met: "The counters are globals; `$( )` runs the function in a subshell, so its `FAILED=1` happens in a copy that disappears. `>/dev/null` just throws away the text; the function still runs in my shell, so the real `FAILED` changes." Not met: "Because the grader redirects output and the variables are separate from output." — describes, no mechanism.

Question 2: `logsleuth`'s suite tests `field_status` directly **and** runs `bin/logsleuth nope` expecting status 2. Name one bug only the black-box test would catch and one only the unit test would pinpoint.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

> **Answer:** **Met:** a black-box test catches wiring/dispatch/option bugs (wrong `case` branch, broken `getopts`, wrong exit code) that unit tests of correct functions never see; a unit test pinpoints *which* function misbehaves (e.g. `field_status` reading field 8) where the black-box test only reports a wrong summary. **Strong:** gives a concrete bug for each side (the two above or equivalents). **Not met:** "black-box tests the whole thing, unit tests the parts" with no bug named.

Hand-in and grading

Hand in `work/assert.sh`, `work/REFLECTION.md` and your final grader output. **Pass** = `0 failed` **and** both reflection answers name the mechanism. Unlimited attempts.

> **Instructor Note:** **Grading procedure (≈5 min per learner).**
>
> 1. `GRADE_TARGET=/path/to/their/work bash labs/lab8-testing/exercise/grade.sh` → require `---- lab8: 4 passed, 0 failed ----`. If `shellcheck` is missing on your machine the last check reports a skipped PASS — certify A-09 only where shellcheck runs.
> 2. Read `REFLECTION.md` against the descriptors. Record `A-09 PASS` only with 4/4 and both answers at Met or better.
> 3. **Acceptable variations:** `(( ++PASSED ))` (passes; note it needs an initialised counter and would abort under `set -u` if the caller forgot — the `${PASSED:-0}` form is the robust one); `echo` instead of `printf` (passes if the text is identical); `[ "$2" = "$3" ]` (passes; quoted, so safe). A version that prints `PASS` lines on success **passes** (stdout of the two matches is not inspected) but is noisier than the course's convention — accept, mention. A version declaring `local PASSED` **fails** — the Lab 4 scope lesson again.
> 4. **Model feedback (Not yet):** "`counted two passes` got `unset`: your function assigns `passed` (lower case) — the grader reads `PASSED`." Remediation: S08.1.
> 5. **Equivalent reassessment:** same grader; Q1′ "Why does `assert_eq` print `got [..] want [..]` in brackets?" (Met: to make leading/trailing spaces and empty strings visible) and Q2′ "Why does the grader use a *deliberate* miss?" (Met: a harness that never fails cannot be shown to count failures; the test of a test is a known failure).

> **Instructor Note:** **Appendix — reference solution (`solution/assert.sh`, observed 4/4):**

```
#!/usr/bin/env bash
# Lab 8 solution — a test harness in a handful of lines: compare, count, report.
# The caller initialises PASSED=0 and FAILED=0 before using assert_eq.

assert_eq() {
  if [[ "$2" == "$3" ]]; then
    PASSED=$(( ${PASSED:-0} + 1 ))
  else
    FAILED=$(( ${FAILED:-0} + 1 ))
    printf 'FAIL %s: got [%s] want [%s]\n' "$1" "$2" "$3"
  fi
}
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.